

Maturidade de sistemas

Por que maturidade agora — e por que não mais CMMI genérico

No Sprint 6 atribuímos nomes formais do BSTQB CTFL v4.0 aos seis artefatos da pasta `internet_banking_tests`. Esse movimento foi de léxico. O Sprint 7 opera em outro nível: avaliação de maturidade do processo de teste como um todo, com instrumentos quantitativos e benchmarks setoriais reais. O CMMI clássico (criado para manufatura em 1991, adaptado para software depois) não é o instrumento adequado aqui — é genérico demais, é de uma era pré-IA e pré-DevOps, e não foi construído para o processo de teste especificamente. Vamos usar três instrumentos contemporâneos que se complementam: TMMi (maturidade específica de teste), DORA 2024 (métricas de entrega de software) e literatura sobre LLM na geração de testes.

Três camadas que se reforçam mutuamente

Camada 1 — TMMi com benchmark setor financeiro: o estudante avalia sua suíte do `internet banking` contra as áreas de processo do TMMi Level 2 e Level 3, usando como referência empírica externa o survey de Van Veenendaal (2024) com 60 instituições financeiras internacionais, que estabeleceu Level 3 "Defined" como o patamar modal do setor.

Camada 2 — DORA 2024 computado sobre dados reais: o estudante calcula numericamente as quatro métricas-chave do Accelerate State of DevOps Report 2024 (Lead Time for Changes, Deployment Frequency, Change Failure Rate, Mean Time to Recovery) sobre o histórico empírico dos seus próprios Sprints 1 a 6 e se posiciona no cluster Low / Medium / High / Elite Performer.

Camada 3 — Position paper crítico em literatura LLM-augmented contemporânea: o estudante lê integralmente El Haji, Brandt e Zaidman (2024) — paper publicado no AST 2024 que estudou exatamente o que fizemos empiricamente — e produz um documento próprio de duas páginas, em formato acadêmico, posicionando-se criticamente sobre lacunas dos modelos de maturidade na era LLM.

Conceito essencial: a virada deste sprint

Nos Sprints 1 a 3 a pergunta foi: o sistema responde corretamente?

No Sprint 4: minha suíte detectaria se ele parasse de responder?

No Sprint 5: meus mutantes sobreviventes são equivalentes ou matáveis?

No Sprint 6: que nome formal cada artefato meu tem na taxonomia do BSTQB?

Neste Sprint 7 a pergunta muda: em que estágio de maturidade do setor financeiro internacional o meu trabalho se posiciona, que números DORA ele produz quando medido com rigor, e que crítica fundamentada eu consigo formular contra os modelos de maturidade vigentes a partir da minha experiência empírica com IA na geração de testes?

Roteiro Sprint 7 — assessment TMMi setor financeiro, DORA 2024 e position paper crítico em literatura AI-augmented

Como ler este roteiro

Este roteiro tem uma lógica de progressão. Cada bloco prepara o seguinte. Você vai encontrar blocos de cinco naturezas, e precisa tratá-los de formas diferentes:

Blocos azuis trazem conceito ou continuidade. Leia com atenção; não há resposta a registrar.

Blocos rosa trazem investigação técnica do aluno. Você precisa registrar sua resposta no arquivo Word de entrega antes de avançar. A resposta é avaliada por consistência com os autores indicados (TMMi Foundation, 2018; VAN VEENENDAAL, 2024; DORA, 2024; EL HAJI; BRANDT; ZAIDMAN, 2024; YUAN et al., 2024; LI et al., 2025) e pela qualidade do raciocínio, não por concordância com nenhum gabarito.

Blocos amarelos trazem atenção técnica ou troubleshooting.

Blocos roxos trazem comandos, prompts ou esqueletos de planilha para adaptar. NUNCA cole o conteúdo bruto — você precisa preencher com seus próprios dados.

Blocos verdes trazem resultado esperado, autoavaliação ou reflexão final. Eles são especialmente importantes — é como você sabe que está no caminho certo sem o professor ao lado.

Atenção: este sprint exige leitura de literatura primária em inglês. Os papers estão em acesso aberto (URLs disponíveis nas Referências). Respostas sem ancoragem nos autores indicados, sem evidência empírica dos seus próprios Sprints 1 a 6, ou construídas como paráfrase superficial dos textos lidos não serão consideradas tecnicamente válidas.

Objetivo deste sprint

Você não vai escrever código novo neste sprint. Vai fazer três coisas que um testador júnior precisa saber fazer no primeiro mês de qualquer emprego sério:

- I) avaliar processo de teste contra um modelo de maturidade real e específico (TMMi),
- II) computar métricas DORA sobre dados reais e posicionar a equipe contra um benchmark internacional, e
- III) ler literatura empírica primária e produzir posicionamento próprio em formato acadêmico. O nível de exigência sobe deliberadamente em relação aos sprints anteriores: aqui não há gabarito, não há resposta consolidada do professor para conferir ao final, e a entrega final tem três artefatos articulados que vão direto para o portfólio profissional e para a apresentação da equipe à banca da Hackathon em 26-06-2026.

Antes de começar — pré-requisitos obrigatórios

[] Sprints 1 a 6 concluídos, com os respectivos arquivos Word de entrega salvos para consulta empírica dos dados próprios.

[] Pasta `internet_banking_tests` intacta, contendo os artefatos consolidados ao final do Sprint 5 (`app.py` refatorado com `config.py`, `conftest.py`, `test_ia_gerado.py`, `test_hypothesis.py`, `test_flask_client.py`, `features/transferencia.feature`, `features/steps/transferencia_steps.py`).

[] Acesso ao AVA (ambiente.virtual.ifro.edu.br) — a aula de 12-06-2026 (sexta-feira) está classificada como Dia Letivo mediado por tecnologia, e o trabalho é integralmente assíncrono no AVA.

[] Capacidade de leitura técnica em inglês para os papers obrigatórios. Os textos estão em acesso aberto via repositório institucional (TU Delft) ou via DOI.

[] Software para cálculo de planilha (LibreOffice Calc, Microsoft Excel ou Google Sheets) — você vai computar quatro métricas numéricas e produzir uma tabela.

Camada 1 — Conceito: TMMi v1.2 e o benchmark setor financeiro

O TMMi (Test Maturity Model integration) é o modelo de maturidade construído especificamente para o processo de teste. Foi originalmente inspirado pelo CMMI, mas o complementa: enquanto o CMMI é genérico para desenvolvimento, o TMMi cobre

rigorosamente apenas a maturidade do teste, com áreas de processo específicas. A versão oficial atualmente em vigor é o TMMi Framework v1.2 (TMMi FOUNDATION, 2018), disponível gratuitamente em <https://www.tmmi.org/tmmi-framework/>.

O TMMi tem cinco níveis de maturidade — Initial, Managed, Defined, Measured, Optimization — cada um com áreas de processo (Process Areas) específicas. As do Level 2 (Managed) são: Test Policy and Strategy, Test Planning, Test Monitoring and Control, Test Design and Execution, Test Environment. As do Level 3 (Defined) incluem entre outras: Test Organization, Test Training Program, Test Lifecycle and Integration, Non-functional Testing e Peer Reviews.

Van Veenendaal (2024) conduziu um survey internacional com 60 instituições financeiras (bancos, seguradoras, fundos de pensão) para mapear a maturidade TMMi alcançada no setor. O achado central foi que o nível modal de maturidade atingido é o TMMi Level 3 "Defined", caracterizado como um nível acima da média. Esse dado é o benchmark externo contra o qual você vai avaliar a sua suíte do internet banking — não como auto-classificação confortável, mas como comparação honesta contra a prática efetiva do setor que o cenário germinal da sua disciplina simula.

Investigação Técnica 1 — auto-avaliação TMMi com referência ao setor financeiro

Para cada uma das cinco áreas de processo do TMMi Level 2, decida com base em evidência empírica dos seus Sprints 1 a 5 se a sua suíte atinge a área de processo ou não. "Atingir" significa que existem práticas observáveis e artefatos concretos que sustentam a afirmação. Não basta dizer "sim, atinge"; é preciso citar o artefato que evidencia.

a) Test Policy and Strategy: há uma política explícita de testes registrada em algum lugar da sua pasta? Atinge ou não atinge? Justifique.

b) Test Planning: há um plano de testes estruturado, ou os testes foram produzidos sprint a sprint sem plano global? Justifique.

c) Test Monitoring and Control: há métricas de progresso de teste sendo monitoradas? (Considere: cobertura, mutation score, número de testes verdes.) Justifique.

d) Test Design and Execution: há técnicas formais de design de teste sendo aplicadas? (Considere: análise de fronteira, partição de equivalência, property-based.) Justifique.

e) Test Environment: há um ambiente de teste controlado e reproduzível? (Considere: conftest.py, fixture autouse, app.config externalizado.) Justifique.

Em seguida, considere o Level 3. Há qualquer evidência empírica de práticas das áreas de processo Test Organization, Test Lifecycle and Integration, Non-functional Testing, Peer Reviews? Provavelmente não — e dizer "não" honestamente é mais valioso do que forjar evidência.

Conclusão honesta: em que nível TMMi a sua suíte se enquadra hoje, e como esse posicionamento se compara ao benchmark Level 3 "Defined" estabelecido por Van Veenendaal (2024) para o setor financeiro? A conclusão é uma frase técnica, com a referência ao paper e ao seu artefato.

Camada 2 — Conceito: DORA 2024 e as quatro métricas-chave

O DORA (DevOps Research and Assessment) é o programa de pesquisa do Google Cloud que publica anualmente o Accelerate State of DevOps Report. A edição de 2024 marca o décimo aniversário do programa e foi construída a partir de mais de 39.000 respondentes globais. O relatório está em acesso público em <https://dora.dev/research/2024/dora-report/>.

O DORA validou empiricamente quatro métricas-chave (the four keys) que medem o desempenho de entrega de software, organizadas em duas dimensões: throughput (velocidade) e stability (estabilidade).

Lead Time for Changes (throughput): tempo entre o commit de uma mudança e seu deploy bem-sucedido em produção.

Deployment Frequency (throughput): quantos deploys de mudanças a equipe realiza por unidade de tempo.

Change Failure Rate (stability): percentual de deploys que causam falhas em produção e exigem hotfix ou rollback.

Mean Time to Recovery / Failed Deployment Recovery Time (stability): tempo médio entre a detecção de uma falha em produção e sua restauração.

Com base nessas quatro métricas, o DORA classifica as equipes em quatro clusters: Low Performer, Medium Performer, High Performer e Elite Performer. Apenas 19% das equipes pesquisadas em 2024 atingiram o cluster Elite (DORA, 2024).

Achado crítico do relatório 2024 — relevante: a adoção de IA na geração de código foi associada a uma queda de 1,5% em throughput e 7,2% em stability nas equipes que adotaram IA mais intensamente. O relatório recomenda cautela e integração metodológica deliberada, não adoção entusiasta sem auditoria. Esse achado faz sentido empírico imediato

à luz do que vocês viveram nos Sprints 1, 4 e 5 com Copilot — testes IA-gerados com lacunas críticas, mutantes sobreviventes onde a IA não viu, e steps Gherkin que violaram a arquitetura.

Investigação Técnica 2 — cálculo numérico das métricas DORA sobre dados próprios

Você vai aplicar as quatro métricas DORA conceitualmente aos dados empíricos dos seus Sprints 1 a 6. Não é metáfora — é cálculo numérico, com unidade. Trate cada "deploy" como a produção de um artefato funcional de teste, e cada "failure in production" como um teste que falhou por defeito real ou por lacuna detectada posteriormente (mutante sobrevivente, gap de cobertura, contraexemplo do Hypothesis).

a) Lead Time for Changes: tempo, em horas ou dias, entre a postagem da sua Q8 (Material de 09-05-2026) e o primeiro pytest verde do `test_ia_gerado.py` no Sprint 1. Compute a partir dos timestamps dos seus arquivos.

b) Deployment Frequency: número de sprints concluídos (com entrega no AVA) por semana ao longo do semestre. Tipicamente um por semana — mas conte com base na sua entrega real.

c) Change Failure Rate: razão entre número de mutantes sobreviventes (Sprint 4) e número total de mutantes gerados pelo mutmut. Esta é uma adaptação semântica do conceito DORA — o mutante sobrevivente representa o equivalente sintético de uma "mudança defeituosa que escapou para produção" porque sua suíte não a detectaria. Expresse como percentual.

d) Mean Time to Recovery: tempo entre a falha do `test_transferencia_saldo_igual_valor` na 2ª execução consecutiva (Sprint 1, HTTP 422 saldo insuficiente) e a correção via `confest.py` com fixture autouse (Sprint 2). Compute em dias ou horas com base no calendário das aulas.

Posicionamento: com base nos quatro números calculados, em qual cluster DORA (Low / Medium / High / Elite) a sua suíte se posicionaria se fosse uma equipe real de produção? Justifique a classificação confrontando seus números com as referências do DORA 2024.

Atividade prática — modelo de planilha DORA

Crie em LibreOffice Calc, Excel ou Google Sheets uma planilha com a estrutura mínima abaixo. Preencha com os seus números reais e exporte como PDF ou imagem para anexar ao arquivo Word de entrega:

Coluna A: Métrica DORA

Coluna B: Definição operacional (uma frase, adaptada ao contexto acadêmico)

Coluna C: Valor numérico calculado sobre os Sprints 1-6 (com unidade)

Coluna D: Cluster sugerido (Low / Medium / High / Elite) e justificativa em uma frase

Linhas a preencher:

1. Lead Time for Changes
2. Deployment Frequency
3. Change Failure Rate
4. Mean Time to Recovery

Os valores absolutos não são o ponto; o que importa é a coerência interna entre as quatro linhas e a honestidade da auto-classificação.

Camada 3 — Conceito: literatura LLM-augmented contemporânea (2024-2025)

Esta camada é onde o estudante de 5º período opera como leitor crítico de literatura primária — não como aluno que recebe definições, mas como quase-profissional que se posiciona em um debate vivo.

Leitura obrigatória: EL HAJI, K.; BRANDT, C.; ZAIDMAN, A. (2024) — “Using GitHub Copilot for Test Generation in Python: An Empirical Study”, publicado nos Proceedings of the 5th ACM/IEEE International Conference on Automation of Software Test (AST 2024), Lisboa, p. 45-55. DOI: 10.1145/3644032.3644443. PDF em acesso aberto via repositório TU Delft.

Esse paper estudou empiricamente exatamente o que vocês fizeram nesta disciplina: geração de testes Python via GitHub Copilot. Os autores analisaram a usabilidade dos testes gerados sobre sete projetos Python open-source e concluíram que a usabilidade é baixa — desenvolvedores precisam fazer modificações na maioria dos testes gerados, e a quantidade de ajustes diminui quando já existem testes na mesma classe que o Copilot pode usar como referência estrutural.

Leitura complementar (escolha uma): (i) YUAN, Z. et al. (2024) — “Evaluating and Improving ChatGPT for Unit Test Generation”, Proceedings of the ACM on Software Engineering, FSE 2024 (DOI: 10.1145/3660783); ou (ii) LI, Y. et al. (2025) — “Evaluating Large Language Models for Software Testing”, Computer Standards & Interfaces, v. 93, 103942 (DOI: 10.1016/j.csi.2024.103942). Ambos investigam de forma sistemática a qualidade e as lacunas de testes gerados por LLM em contextos diversos.

Investigação Técnica 3 — confronto entre achados empíricos e experiência própria

Após ler integralmente El Haji, Brandt e Zaidman (2024), responda no seu arquivo Word com referência a página ou seção sempre que possível:

a) Os autores observaram que a usabilidade dos testes gerados pelo Copilot é baixa. Que evidência empírica dos seus Sprints 1, 4 e 5 confirma esse achado? Cite especificamente: o que faltou no `test_ia_gerado.py` do Sprint 1 que você teve que corrigir no Sprint 2? Que problema arquitetural do Copilot apareceu nos steps gerados do Sprint 5?

b) Os autores observaram que a qualidade dos testes gerados melhora quando já existem testes na mesma classe a serem mimetizados. Como esse achado se relaciona com a sua experiência de gerar `confest.py` no Sprint 2 a partir da sua observação crítica do Sprint 1?

c) Os autores tratam o Copilot como ferramenta de assistência, não como autoridade. Em que momento dos seus seis sprints você sentiu — empiricamente — que o Copilot era uma autoridade que não devia ter sido tratada como autoridade?

Investigação Técnica 4 — produção do position paper crítico (entrega central do sprint)

Escreva um documento próprio de duas páginas, em formato acadêmico (Times New Roman 12, espaçamento 1,5, justificado, citações no padrão ABNT), com a estrutura mínima abaixo:

Título: uma tese curta, defensável e específica. Não use "Reflexões sobre IA e teste". Use algo como "A área de processo X do TMMi precisa ser reformulada para incluir Y na era LLM" ou "A métrica DORA Lead Time for Changes torna-se ambígua quando Z".

Parágrafo 1 — Contexto e tese (até 8 linhas): contextualize o problema em uma frase, anuncie sua tese explicitamente.

Parágrafo 2 — Evidência empírica própria (até 15 linhas): narre dois ou três episódios concretos dos seus Sprints 1 a 6 que sustentam a tese. Cite os artefatos por nome.

Parágrafo 3 — Diálogo crítico com a literatura (até 15 linhas): compare seus achados com El Haji et al. (2024) e com Yuan et al. (2024) ou Li et al. (2025). Concorda? Discorda? Em que ponto?

Parágrafo 4 — Proposta (até 12 linhas): que reformulação concreta você propõe? Pode ser uma nova área de processo TMMi, uma extensão de métrica DORA, um novo princípio de auditoria de IA, etc. Seja específico — "deveria haver mais cuidado com IA" não é proposta.

Parágrafo 5 — Limites do argumento (até 8 linhas): onde sua tese é frágil? Que evidência adicional seria necessária para sustentá-la de forma mais robusta? Reconhecer limites é sinal de maturidade acadêmica, não de fraqueza.

Referências: no mínimo cinco entradas em ABNT, das quais pelo menos três publicadas em 2023-2025.

Critério de avaliação do position paper: (i) clareza e especificidade da tese; (ii) qualidade da evidência empírica própria; (iii) profundidade do diálogo crítico com a literatura (não paráfrase); (iv) concretude e viabilidade da proposta; (v) honestidade do reconhecimento de limites; (vi) qualidade do português acadêmico.

Atenção — armadilhas comuns neste sprint

Primeiro: não tente parafrasear o paper do El Haji et al. — o avaliador vai pedir a sua tese, não o resumo do paper deles. Segundo: não invente número DORA. Se você não consegue computar o Lead Time porque não anotou os timestamps, registre honestamente que o dado é estimado e justifique. Terceiro: o TMMi Level 5 "Optimization" não é alcançável em um semestre acadêmico — não tente reivindicá-lo. Quarto: o cluster Elite DORA também não — apenas 19% das equipes profissionais em 2024 atingiram. Modéstia diagnóstica vale mais do que ambição infundada.

Síntese epistemológica — registre em seu arquivo Word

Responda em três frases curtas, antes do position paper final:

- a) Qual é a diferença operacional, para você, entre "saber o nome do que produzi" (Sprint 6) e "saber o nível de maturidade do que produzi" (Sprint 7)?
- b) Que número DORA mais lhe surpreendeu ao calcular sobre os seus dados, e por quê?
- c) Em uma frase, qual é a sua tese para o position paper — antes mesmo de escrevê-lo?

Entrega no AVA

Arquivo Word ou PDF único contendo, na ordem:

- 1) Auto-avaliação TMMi: cinco áreas de processo do Level 2 avaliadas com evidência empírica, conclusão sobre o nível alcançado, e comparação honesta com o benchmark Level 3 do setor financeiro (Van Veenendaal, 2024).
- 2) Suas respostas à Investigação Técnica 2 (cálculo das quatro métricas DORA) e a planilha DORA exportada como imagem ou PDF anexado.
- 3) Suas respostas à Investigação Técnica 3 (confronto com achados de El Haji et al., 2024) em três itens articulados.
- 4) Position paper crítico de duas páginas (Investigação Técnica 4), em formato acadêmico, com referências em ABNT.
- 5) Síntese epistemológica em três frases.

Pontuação: até 10 pontos. A distribuição é: 2 pontos pela auto-avaliação TMMi com evidência empírica; 2 pontos pelo cálculo DORA correto e pela planilha; 1 ponto pelo confronto crítico com El Haji et al.; 4 pontos pelo position paper (clareza da tese, evidência, diálogo com literatura, concretude da proposta, reconhecimento de limites); 1 ponto pela síntese epistemológica.

Respostas vagas, paráfrase superficial dos textos lidos, números DORA forjados sem base ou tese ausente no position paper não serão consideradas tecnicamente válidas.

Referências Bibliográficas

DORA / GOOGLE CLOUD. *Accelerate State of DevOps Report 2024*. Google Cloud, 2024. Disponível em: <https://dora.dev/research/2024/dora-report/>. Acesso em: junho 2026.

EL HAJI, K.; BRANDT, C.; ZAIDMAN, A. Using GitHub Copilot for Test Generation in Python: An Empirical Study. In: ACM/IEEE INTERNATIONAL CONFERENCE ON AUTOMATION OF SOFTWARE TEST, 5., 2024, Lisbon. *Proceedings...* New York: ACM, 2024. p. 45-55. DOI: 10.1145/3644032.3644443. Disponível em: <https://doi.org/10.1145/3644032.3644443>. Versão em acesso aberto: https://repository.tudelft.nl/file/File_c22470cb-5514-4537-ab3e-19b8c37ebc8f. Acesso em: junho 2026.

LI, Y. et al. Evaluating large language models for software testing. *Computer Standards & Interfaces*, v. 93, 103942, 2025. DOI: 10.1016/j.csi.2024.103942. Disponível em: <https://doi.org/10.1016/j.csi.2024.103942>. Acesso em: junho 2026.

TMMi FOUNDATION. *TMMi Framework*: Release 1.2. TMMi Foundation, 2018. Disponível em: <https://www.tmmi.org/tmmi-framework/>. Acesso em: junho 2026.

VAN VEENENDAAL, E. Test Maturity Model integration (TMMi): Test Maturity in the Financial Domain. *American Journal of Computer Science and Technology*, v. 7, n. 2, 2024. DOI: 10.11648/j.ajcst.20240702.13. Disponível em: <https://www.sciencepublishinggroup.com/article/10.11648/j.ajcst.20240702.13>. Acesso em: junho 2026.

YUAN, Z. et al. Evaluating and Improving ChatGPT for Unit Test Generation. *Proceedings of the ACM on Software Engineering*, v. 1, n. FSE, 2024. DOI: 10.1145/3660783. Disponível em: <https://doi.org/10.1145/3660783>. Acesso em: junho 2026.

Referências legadas dos Sprints 1 a 6 — mantidas como contexto

BSTQB — BRAZILIAN SOFTWARE TESTING QUALIFICATIONS BOARD. *Certified Tester Foundation Level Syllabus*: versão 4.0. [S. l.]: ISTQB; BSTQB, 2023. Disponível em: https://www.bstqb.online/files/syllabus_ctfl_4.0br.pdf. Acesso em: maio 2026.